

Metalean

Draft.

1 Syntax

Definition 1 (Levels).

$$l \in \text{Level} ::= u \mid 0 \mid l + 1 \mid \max(l, l) \mid \text{imax}(l, l)$$

Definition 2 (Semantics of levels).

$$\begin{aligned} \llbracket - \rrbracket_- : \text{Level} &\rightarrow (\mathbb{N} \rightarrow \mathbb{N}) \rightarrow \mathbb{N} \\ \llbracket u \rrbracket_\nu &= \nu(u) \\ \llbracket 0 \rrbracket_\nu &= 0 \\ \llbracket l + 1 \rrbracket_\nu &= \llbracket l \rrbracket_\nu + 1 \\ \llbracket \max(l_1, l_2) \rrbracket_\nu &= \max(\llbracket l_1 \rrbracket_\nu, \llbracket l_2 \rrbracket_\nu) \\ \llbracket \text{imax}(l_1, l_2) \rrbracket_\nu &= \begin{cases} 0 & \text{if } \llbracket l_2 \rrbracket_\nu = 0 \\ \max(\llbracket l_1 \rrbracket_\nu, \llbracket l_2 \rrbracket_\nu) & \text{otherwise} \end{cases} \\ l_1 = l_2 &\iff \forall \nu. \llbracket l_1 \rrbracket_\nu = \llbracket l_2 \rrbracket_\nu \\ l_1 \leq l_2 &\iff \forall \nu. \llbracket l_1 \rrbracket_\nu \leq \llbracket l_2 \rrbracket_\nu \end{aligned}$$

Definition 3 (Expressions).

$$\begin{aligned} e, t \in \text{Expr} ::= & x \mid \text{Sort}_l \mid c.\{\bar{l}\} \\ & \mid \forall x : t. t \mid \lambda x : t. e \mid e e \mid \text{letE } x : t := e; e \\ & \mid \text{ind}(I, s, \bar{l}, \bar{p}, \bar{i}) \mid I_{\text{ctor}}^{s,c}.\{\bar{l}\}(\bar{p}, \bar{a}, \bar{r}) \mid I_{\text{rec}}^s.\{\bar{l}, l_\mu\}(\bar{p}, \bar{\mu}, \bar{m}, \bar{i}, e) \\ & \mid Q.\{l\}(\alpha, r) \mid Q_{\text{mk}}.\{l\}(\alpha, r, a) \mid Q_{\text{lift}}.\{l_\alpha, l_\beta\}(\alpha, r, \beta, f, h, a) \mid Q_{\text{ind}}.\{l\}(\alpha, r, \beta, f, a) \end{aligned}$$

We use names for convenience, but use de Bruijn indices in practice.

We sometimes write $\forall x : t_1. t_2$ in the following ways: $\Pi x : t_1. t_2$, $(x : t_1) \rightarrow t_2$, and $t_1 \rightarrow t_2$ (when x does not occur in t_2).

Definition 4 (Universe abbreviations).

$$\text{Prop} := \text{Sort}_0 \qquad \text{Type}_l := \text{Sort}_{l+1}$$

Definition 5 (Contexts).

$$\Gamma, \Delta \in \text{Ctx} ::= \cdot \mid \Gamma, x : t$$

We write $(\bar{x} : \Gamma) \rightarrow t$ to mean the multiple abstraction of the type t given by

$$(x_1 : \Gamma_1) \rightarrow \dots \rightarrow (x_n : \Gamma_n) \rightarrow t.$$

Definition 6 (Environments).

$$\begin{aligned}
E &\in \text{Env} ::= \cdot \mid E, D \\
D &\in \text{Entry} ::= \mathbf{axiom} \ c.\{\bar{u}\} : t \\
&\quad \mid \mathbf{opaque} \ c.\{\bar{u}\} : t \\
&\quad \mid \mathbf{def} \ c.\{\bar{u}\} : t := e \\
&\quad \mid \mathbf{mutual inductive} \ I \dots \mathbf{end} \\
&\quad \mid \mathbf{init_quot}(I_{\text{Eq}})
\end{aligned}$$

Definition 7 (Inductive types).

$$\begin{aligned}
&\mathbf{mutual} \\
&\quad \mathbf{inductive} \ I^1.\{\bar{u}\} \ (\bar{p} : \Gamma_{\text{param}}) : \Gamma_{\text{idx}}^1 \rightarrow \text{Sort}_l \ \mathbf{where} \\
&\quad \mid I_{\text{ctor}}^{1,1} : \Gamma_{\text{field}}^{1,1} \rightarrow \bar{R}^{1,1} \rightarrow \text{ind}(I, 1, \bar{u}, \bar{p}, \bar{i}^{1,1}) \\
&\quad \vdots \\
&\quad \mid I_{\text{ctor}}^{1, n_{\text{ctor}}^1} : \Gamma_{\text{field}}^{1, n_{\text{ctor}}^1} \rightarrow \bar{R}^{1, n_{\text{ctor}}^1} \rightarrow \text{ind}(I, 1, \bar{u}, \bar{p}, \bar{i}^{1, n_{\text{ctor}}^1}) \\
&\quad \vdots \\
&\quad \mathbf{inductive} \ I^{n_{\text{sort}}}.\{\bar{u}\} \ (\bar{p} : \Gamma_{\text{param}}) : \Gamma_{\text{idx}}^{n_{\text{sort}}} \rightarrow \text{Sort}_l \ \mathbf{where} \\
&\quad \mid I_{\text{ctor}}^{n_{\text{sort}}, 1} : \Gamma_{\text{field}}^{n_{\text{sort}}, 1} \rightarrow \bar{R}^{n_{\text{sort}}, 1} \rightarrow \text{ind}(I, n_{\text{sort}}, \bar{u}, \bar{p}, \bar{i}^{n_{\text{sort}}, 1}) \\
&\quad \vdots \\
&\quad \mid I_{\text{ctor}}^{n_{\text{sort}}, n_{\text{ctor}}^{n_{\text{sort}}}} : \Gamma_{\text{field}}^{n_{\text{sort}}, n_{\text{ctor}}^{n_{\text{sort}}}} \rightarrow \bar{R}^{n_{\text{sort}}, n_{\text{ctor}}^{n_{\text{sort}}}} \rightarrow \text{ind}(I, n_{\text{sort}}, \bar{u}, \bar{p}, \bar{i}^{n_{\text{sort}}, n_{\text{ctor}}^{n_{\text{sort}}}}) \\
&\mathbf{end} \\
&\Gamma_{\text{field}}^{s,c} = (a_1 : A_1, \dots, a_m : A_m) \quad \text{with levels} \quad v_1, \dots, v_m \\
&\bar{R}^{s,c} = (R^{s,c,1}, \dots, R^{s,c,n_{\text{rec}}^{s,c}}) \\
&R^{s,c,g} = \Gamma_{\text{rec}}^{s,c,g} \rightarrow \text{ind}(I, s^{s,c,g}, \bar{u}, \bar{p}, \bar{j}^{s,c,g})
\end{aligned}$$

This is the full generality of inductive types!¹ Let's take it apart. First, let us regard every inductive type as belonging to some *mutual* inductive “block”. Even if it is not mutually inductive, we can regard it as being mutual with only itself. We will omit the **mutual** and **end** keywords in such a case. We call the mutual inductive's comprising inductives its “sorts” (not to be confused with Sort_u !), and we let the variable $1 \leq s \leq n_{\text{sort}}$ range over the possible sorts of an inductive with n_{sort} such sorts. While a mutual inductive with zero sorts is representable, this is not really an issue.² Here, I refers to the “name” of the *entire* mutual block. In Lean, mutual blocks are not given names themselves, but rather, the individual sorts are given names. Here, we write I^s to refer to the s th sort of a the mutual block with name I .

In Lean, every mutual inductive block comes with some universe variables $\{\bar{u}\}$, parameters $(\bar{p} : \Gamma_{\text{param}})$ forming a dependent telescope, and a resulting universe type. These are fixed over all sorts. Additionally, each sort contains some *indices*, also forming a dependent telescope.

¹Except of course, *nested* inductive types.

²Because no expression could possibly refer to a mutual inductive with zero sorts.

Example 8 (Indexed vectors).

inductive Vector. $\{u\}$ ($\alpha : \text{Type}_u$) : ($n : \text{Nat}$) $\rightarrow$ Prop **where**
 | nil : Vector. $\{u\}$ (α , 0)
 | cons : ($n : \text{Nat}$) $\rightarrow$ $\alpha \rightarrow$ Vector. $\{u\}$ (α , n) $\rightarrow$ Vector. $\{u\}$ (α , $n + 1$)

Here, ($\alpha : \text{Type}_u$) is a parameter, and ($n : \text{Nat}$) is an index.

Example 9 (Accessibility).

inductive Acc. $\{u\}$ ($\alpha : \text{Sort}_u$) ($r : \alpha \rightarrow \alpha \rightarrow \text{Prop}$) : ($x : \alpha$) $\rightarrow$ Prop **where**
 | intro : ($x : \alpha$) $\rightarrow$ ($\forall y : \alpha. r \ y \ x \rightarrow \text{Acc.}\{u\}(\alpha, r, y)$) $\rightarrow$ Acc. $\{u\}(\alpha, r, x)$

Here, the recursive field is accompanied by a telescope ($y : \alpha$) $\rightarrow$ ($h : r \ y \ x$) $\rightarrow$ $-$.

Each constructor $I_{\text{ctor}}^{s,c}$ has three components.

The *ordinary fields* $\Gamma_{\text{field}}^{s,c}$ form a dependent telescope over the parameters. Each field may mention the parameters and the fields before it, but not the inductive being defined. In cons, the ordinary fields are ($n : \text{Nat}$) and the α .

The *recursive fields* $\bar{R}^{s,c}$ are the only place where the block refers to itself. Each $R^{s,c,g}$ has its own telescope of arguments $\Gamma_{\text{rec}}^{s,c,g}$, which may mention the parameters and all of the ordinary fields, and ends in some sort $s_{\text{rec}}^{s,c,g}$ of the same block, at the same universe variables $\{\bar{u}\}$ and parameters $\bar{p}$, with indices $\bar{j}^{s,c,g}$. Strict positivity holds by construction! Why do we say that recursive fields come last? It is actually impossible for a recursive field to be *depended* on without violating strict positivity, so the recursive fields are independent of one another and form a tuple, and we may assume that they all come after the ordinary fields.

Proposition 10 (Recursive fields cannot be depended on usefully). *Let $x : (\bar{y} : \Gamma_{\text{rec}}) \rightarrow \text{ind}(I, s, \bar{u}, \bar{p}, \bar{j})$ be a recursive field of a constructor, and let t be the type of a later field, in β -normal form. If x occurs in t , then I occurs in t at a negative position.*

Proof. A term whose type ends in I can only be used by passing it to something whose domain ends in I .³ Nothing in scope has such a domain: the types of constants, parameters, non-recursive fields and I itself cannot mention I , and positivity forbids it in the domains of recursive fields. So wherever x occurs in t , the domain it is passed to must acquire I from t itself, which puts I in t at a negative position. $\square$

Why did we require that t is in β -normal form? consider the following inductive type, which is accepted by Lean:

inductive N : Type₀ **where**
 | mk : ($x : \text{N}$) $\rightarrow$ ($\lambda x : \text{N}. \text{Nat}$) $x \rightarrow \text{N}$

N occurs negatively, but it does not occur negatively in a *useful* way, which is evident once the β -redex is reduced. It follows that, if x occurs in t , not in β -normal form, β -normalising t deletes x , showing that x was never really useful in the first place.

Despite this fact, Lean allows non-recursive and recursive fields to be interleaved. For simplicity, our syntax does *not* allow interleaving, and outright *disallows* recursive fields to be depended on. To be able to consume the same input as Lean, we perform a frontend translation that

³By no-confusion, to be proven!

sifts all recursive fields to the end, thereby weakening the contexts in which their types were defined in.

The *target indices* $\bar{i}^{s,c}$ give the indices at which the constructor lands. They may mention the parameters and the ordinary fields, but not the recursive fields. In cons, the target index is $n + 1$, while the recursive field has index n .

Example 11 (Even and odd).

```

mutual
  inductive Even : (n : Nat) → Prop where
    | zero : Even(0)
    | succ : (n : Nat) → Odd(n) → Even(n + 1)
  inductive Odd : (n : Nat) → Prop where
    | succ : (n : Nat) → Even(n) → Odd(n + 1)
end

```

Here Even and Odd are the two sorts of one block. The recursive field of Even.succ lands in Odd, and that of Odd.succ lands in Even.

2 Typing

Definition 12 (Substitution and instantiation). We write $e[\bar{a}/\bar{x}]$ for simultaneous capture-avoiding substitution and $e\{\bar{l}/\bar{u}\}$ for the instantiation of universe variables. These are defined in the obvious way and so their definitions are not given here. We write l_I for the resulting level of I .

$E[\Gamma] \vdash e_1 \equiv e_2 : t \quad (\text{definitional equality})$

The judgement is relative to a fixed list of universe variables $\bar{u}$, over which the levels in Γ , e_1 , e_2 and t range. We define the usual typing $E[\Gamma] \vdash e : t$ as the diagonal $E[\Gamma] \vdash e \equiv e : t$. For $\Delta = (x_1 : t_1, \dots, x_n : t_n)$, $E[\Gamma] \vdash \bar{a} \equiv \bar{b} : \Delta$ abbreviates the premises $E[\Gamma] \vdash a_k \equiv b_k : t_k[a_1, \dots, a_{k-1}/x_1, \dots, x_{k-1}]$ for each k , and $E[\Gamma] \vdash \bar{a} : \Delta$ abbreviates $E[\Gamma] \vdash \bar{a} \equiv \bar{a} : \Delta$.

$$\begin{array}{c}
\frac{E[\Gamma] \vdash e_1 \equiv e_2 : t}{E[\Gamma] \vdash e_2 \equiv e_1 : t} \text{SYMM} \qquad \frac{E[\Gamma] \vdash e_1 \equiv e_2 : t \quad E[\Gamma] \vdash e_2 \equiv e_3 : t}{E[\Gamma] \vdash e_1 \equiv e_3 : t} \text{TRANS} \\
\\
\frac{(x : t) \in \Gamma \quad E[\Gamma] \vdash t : \text{Sort}_{l_I}}{E[\Gamma] \vdash x : t} \text{VAR} \qquad \frac{E[\Gamma] \vdash t_1 \equiv t_2 : \text{Sort}_{l_I} \quad E[\Gamma] \vdash e_1 \equiv e_2 : t_1}{E[\Gamma] \vdash e_1 \equiv e_2 : t_2} \text{CONV} \\
\\
\frac{}{E[\Gamma] \vdash \text{Sort}_{l_I} : \text{Sort}_{l_{I+1}}} \text{SORT} \\
\\
\frac{([\text{def}, \text{opaque}, \text{axiom}]_c.\{\bar{u}\} : t) \in E \quad E[\Gamma] \vdash t\{\bar{l}/\bar{u}\} : \text{Sort}_{l_v}}{E[\Gamma] \vdash c.\{\bar{l}\} : t\{\bar{l}/\bar{u}\}} \text{CONST}
\end{array}$$

$$\begin{array}{c}
\frac{(\mathbf{def} \ c.\{\bar{u}\} : t := e) \in \mathbf{E} \quad \mathbf{E}[\Gamma] \vdash t\{\bar{l}/\bar{u}\} : \text{Sort}_v \quad \mathbf{E}[\Gamma] \vdash e\{\bar{l}/\bar{u}\} : t\{\bar{l}/\bar{u}\}}{\mathbf{E}[\Gamma] \vdash c.\{\bar{l}\} \equiv e\{\bar{l}/\bar{u}\} : t\{\bar{l}/\bar{u}\}} \text{CONST-}\delta \\
\\
\frac{\mathbf{E}[\Gamma] \vdash t_1 \equiv t_2 : \text{Sort}_{l_1} \quad \mathbf{E}[\Gamma, x : t_1] \vdash t'_1 \equiv t'_2 : \text{Sort}_{l_2} \quad \mathbf{E}[\Gamma, x : t_2] \vdash t'_1 \equiv t'_2 : \text{Sort}_{l_2}}{\mathbf{E}[\Gamma] \vdash \forall x : t_1.t'_1 \equiv \forall x : t_2.t'_2 : \text{Sort}_{\text{imax}(l_1, l_2)}} \text{V-FORM} \\
\\
\frac{\mathbf{E}[\Gamma, x : t_1] \vdash t' : \text{Sort}_{l_2} \quad \mathbf{E}[\Gamma, x : t_1] \vdash e_1 \equiv e_2 : t' \quad \mathbf{E}[\Gamma] \vdash t_1 \equiv t_2 : \text{Sort}_{l_1} \quad \mathbf{E}[\Gamma, x : t_2] \vdash t' : \text{Sort}_{l_2} \quad \mathbf{E}[\Gamma, x : t_2] \vdash e_1 \equiv e_2 : t'}{\mathbf{E}[\Gamma] \vdash \lambda x : t_1.e_1 \equiv \lambda x : t_2.e_2 : \forall x : t_1.t'} \text{V-INTRO} \\
\\
\frac{\mathbf{E}[\Gamma] \vdash t : \text{Sort}_{l_1} \quad \mathbf{E}[\Gamma] \vdash f_1 \equiv f_2 : \forall x : t.t' \quad \mathbf{E}[\Gamma, x : t] \vdash t' : \text{Sort}_{l_2} \quad \mathbf{E}[\Gamma] \vdash a_1 \equiv a_2 : t \quad \mathbf{E}[\Gamma] \vdash t'[a_1/x] \equiv t'[a_2/x] : \text{Sort}_{l_2}}{\mathbf{E}[\Gamma] \vdash f_1 a_1 \equiv f_2 a_2 : t'[a_1/x]} \text{V-ELIM} \\
\\
\frac{\mathbf{E}[\Gamma] \vdash t : \text{Sort}_{l_1} \quad \mathbf{E}[\Gamma] \vdash e : t \quad \mathbf{E}[\Gamma, x : t] \vdash t' : \text{Sort}_{l_2} \quad \mathbf{E}[\Gamma] \vdash t'[e/x] : \text{Sort}_{l_2} \quad \mathbf{E}[\Gamma, x : t] \vdash e' : t' \quad \mathbf{E}[\Gamma] \vdash e'[e/x] : t'[e/x]}{\mathbf{E}[\Gamma] \vdash (\lambda x : t.e') e \equiv e'[e/x] : t'[e/x]} \text{V-}\beta \\
\\
\frac{\mathbf{E}[\Gamma] \vdash t : \text{Sort}_{l_1} \quad \mathbf{E}[\Gamma, x : t] \vdash t' : \text{Sort}_{l_2} \quad \mathbf{E}[\Gamma, x : t] \vdash e : \forall y : t.t'[y/x] \quad \mathbf{E}[\Gamma, x : t] \vdash t : \text{Sort}_{l_1} \quad \mathbf{E}[\Gamma] \vdash e : \forall x : t.t'}{\mathbf{E}[\Gamma] \vdash \lambda x : t.e x \equiv e : \forall x : t.t'} \text{V-}\eta \\
\\
\frac{\mathbf{E}[\Gamma] \vdash t : \text{Sort}_{l_1} \quad \mathbf{E}[\Gamma] \vdash v : t \quad \mathbf{E}[\Gamma] \vdash r : \text{Sort}_{l_2} \quad \mathbf{E}[\Gamma] \vdash e'[v/x] : r}{\mathbf{E}[\Gamma] \vdash \mathbf{let} \mathbf{E} \ x : t := v; e' \equiv e'[v/x] : r} \text{LET-}\zeta^4 \\
\\
\frac{\mathbf{E}[\Gamma] \vdash p : \text{Prop} \quad \mathbf{E}[\Gamma] \vdash h_1 : p \quad \mathbf{E}[\Gamma] \vdash h_2 : p}{\mathbf{E}[\Gamma] \vdash h_1 \equiv h_2 : p} \text{PROOF-IRREL} \\
\\
\frac{I \in \mathbf{E} \quad \mathbf{E}[\Gamma] \vdash \bar{p}_1 \equiv \bar{p}_2 : \Gamma_{\text{param}} \quad \mathbf{E}[\Gamma] \vdash \bar{i}_1 \equiv \bar{i}_2 : \Gamma_{\text{idx}}^s(\bar{p}_1)}{\mathbf{E}[\Gamma] \vdash I^s.\{\bar{l}\}(\bar{p}_1, \bar{i}_1) \equiv I^s.\{\bar{l}\}(\bar{p}_2, \bar{i}_2) : \text{Sort}_{l_I\{\bar{l}/\bar{u}\}}} \text{IND-FORM}
\end{array}$$

⁴Congruence for let-expressions is derivable

$$\begin{array}{l}
I \in E \\
E[\Gamma] \vdash \bar{p}_1 \equiv \bar{p}_2 : \Gamma_{\text{param}} \\
E[\Gamma] \vdash \bar{a}_1 \equiv \bar{a}_2 : \Gamma_{\text{field}}^{s,c}(\bar{p}_1) \\
\forall g. E[\Gamma] \vdash r_{1,g} \equiv r_{2,g} : R^{s,c,g}(\bar{p}_1, \bar{a}_1) \\
\forall f. E[\Gamma] \vdash A_f(\bar{p}_1, \bar{a}_1) \equiv A_f(\bar{p}_2, \bar{a}_2) : \text{Sort}_{w_f} \\
\forall g. E[\Gamma] \vdash R^{s,c,g}(\bar{p}_1, \bar{a}_1) \equiv R^{s,c,g}(\bar{p}_2, \bar{a}_2) : \text{Sort}_{w'_g} \\
E[\Gamma] \vdash I^s \cdot \{\bar{l}\}(\bar{p}_1, \bar{i}^{s,c}(\bar{p}_1, \bar{a}_1)) \equiv I^s \cdot \{\bar{l}\}(\bar{p}_2, \bar{i}^{s,c}(\bar{p}_2, \bar{a}_2)) : \text{Sort}_{l_I \{\bar{l}/\bar{u}\}} \\
\hline
E[\Gamma] \vdash I_{\text{ctor}}^{s,c} \cdot \{\bar{l}\}(\bar{p}_1, \bar{a}_1, \bar{r}_1) \equiv I_{\text{ctor}}^{s,c} \cdot \{\bar{l}\}(\bar{p}_2, \bar{a}_2, \bar{r}_2) : I^s \cdot \{\bar{l}\}(\bar{p}_1, \bar{i}^{s,c}(\bar{p}_1, \bar{a}_1)) \quad \text{IND-INTRO}
\end{array}$$

LARGE ELIMINATION

$$\begin{array}{c}
\frac{1 \leq l_I}{\vdash I^s \text{ large}} \text{ LARGE} \\
\\
\frac{n_{\text{ctor}}^s \leq 1 \quad \forall c, f. v_f^{s,c} = 0 \vee \exists k. i_k^{s,c} = a_f \quad \forall c. n_{\text{rec}}^{s,c} = 0 \vee l_I = 0}{\vdash I^s \text{ large}} \text{ SUBSINGLETON}^5
\end{array}$$

$$\begin{array}{l}
I \in E \\
v = 0 \quad \vee \quad \forall s'. \vdash I^{s'} \text{ large} \\
E[\Gamma] \vdash \bar{p}_1 \equiv \bar{p}_2 : \Gamma_{\text{param}} \\
\forall s'. E[\Gamma] \vdash \mu_1^{s'} \equiv \mu_2^{s'} : (\bar{i} : \Gamma_{\text{idx}}^{s'}(\bar{p}_1)) \rightarrow I^{s'} \cdot \{\bar{l}\}(\bar{p}_1, \bar{i}) \rightarrow \text{Sort}_v \\
\forall s', c. E[\Gamma] \vdash m_1^{s',c} \equiv m_2^{s',c} : (\bar{a} : \Gamma_{\text{field}}^{s',c}(\bar{p}_1)) \rightarrow (\bar{r} : \bar{R}^{s',c}(\bar{p}_1, \bar{a})) \\
\rightarrow \left((\bar{y} : \Gamma_{\text{rec}}^{s',c,g}(\bar{p}_1, \bar{a})) \rightarrow \mu_1^{s_{\text{rec}}^{s',c,g}} \bar{j}^{s',c,g}(\bar{p}_1, \bar{a}) r_g \bar{y} \right)_g \\
\rightarrow \mu_1^{s'} \bar{i}^{s',c}(\bar{p}_1, \bar{a}) I_{\text{ctor}}^{s',c} \cdot \{\bar{l}\}(\bar{p}_1, \bar{a}, \bar{r}) \\
E[\Gamma] \vdash \bar{i}_1 \equiv \bar{i}_2 : \Gamma_{\text{idx}}^s(\bar{p}_1) \\
E[\Gamma] \vdash x_1 \equiv x_2 : I^s \cdot \{\bar{l}\}(\bar{p}_1, \bar{i}_1) \\
E[\Gamma] \vdash \mu_1^s \bar{i}_1 x_1 \equiv \mu_2^s \bar{i}_2 x_2 : \text{Sort}_v \\
\hline
E[\Gamma] \vdash I_{\text{rec}}^s \cdot \{\bar{l}, v\}(\bar{p}_1, \bar{\mu}_1, \bar{\bar{m}}_1, \bar{i}_1, x_1) \equiv I_{\text{rec}}^s \cdot \{\bar{l}, v\}(\bar{p}_2, \bar{\mu}_2, \bar{\bar{m}}_2, \bar{i}_2, x_2) : \mu_1^s \bar{i}_1 x_1 \quad \text{IND-ELIM}
\end{array}$$

ι -REDUCTION

$$\begin{array}{l}
\text{lhs}^{s,c} = I_{\text{rec}}^s \cdot \{\bar{l}, v\}(\bar{p}, \bar{\mu}, \bar{\bar{m}}, \bar{i}^{s,c}(\bar{p}, \bar{a}), I_{\text{ctor}}^{s,c} \cdot \{\bar{l}\}(\bar{p}, \bar{a}, \bar{r})) \\
\text{rhs}^{s,c} = m^{s,c} \bar{a} \bar{r} \left(\lambda \bar{y} : \Gamma_{\text{rec}}^{s,c,g}(\bar{p}, \bar{a}). I_{\text{rec}}^{s_{\text{rec}}^{s,c,g}} \cdot \{\bar{l}, v\}(\bar{p}, \bar{\mu}, \bar{\bar{m}}, \bar{j}^{s,c,g}(\bar{p}, \bar{a}), r_g \bar{y}) \right)_g \\
T^{s,c} = \mu^s \bar{i}^{s,c}(\bar{p}, \bar{a}) I_{\text{ctor}}^{s,c} \cdot \{\bar{l}\}(\bar{p}, \bar{a}, \bar{r})
\end{array}$$

⁵Lean *also* requires $n_{\text{sort}} = 1$, but this is not actually necessary, so we omit it here.

$$\begin{array}{l}
I \in \mathbf{E} \\
v = 0 \vee \forall s'. \vdash I^{s'} \text{ large} \\
\mathbf{E}[\Gamma] \vdash \bar{p} : \Gamma_{\text{param}} \\
\forall s'. \mathbf{E}[\Gamma] \vdash \mu^{s'} : (\bar{i} : \Gamma_{\text{idx}}^{s'}(\bar{p})) \rightarrow I^{s'}. \{\bar{l}\}(\bar{p}, \bar{i}) \rightarrow \text{Sort}_v \\
\forall s', c'. \mathbf{E}[\Gamma] \vdash m^{s', c'} : (\bar{a} : \Gamma_{\text{field}}^{s', c'}(\bar{p})) \rightarrow (\bar{r} : \bar{R}^{s', c'}(\bar{p}, \bar{a})) \\
\rightarrow \left((\bar{y} : \Gamma_{\text{rec}}^{s', c', g}(\bar{p}, \bar{a})) \rightarrow \mu_{\text{rec}}^{s', c', g} \bar{j}^{s', c', g}(\bar{p}, \bar{a}) r_g \bar{y} \right)_g \\
\rightarrow \mu^{s'} \bar{i}^{s', c'}(\bar{p}, \bar{a}) I_{\text{ctor}}^{s', c'}. \{\bar{l}\}(\bar{p}, \bar{a}, \bar{r}) \\
\mathbf{E}[\Gamma] \vdash \bar{a} : \Gamma_{\text{field}}^{s, c}(\bar{p}) \\
\forall g. \mathbf{E}[\Gamma] \vdash r_g : R^{s, c, g}(\bar{p}, \bar{a}) \\
\mathbf{E}[\Gamma] \vdash T^{s, c} : \text{Sort}_v \\
\mathbf{E}[\Gamma] \vdash \text{lhs}^{s, c} : T^{s, c} \\
\mathbf{E}[\Gamma] \vdash \text{rhs}^{s, c} : T^{s, c} \\
\hline
\mathbf{E}[\Gamma] \vdash \text{lhs}^{s, c} \equiv \text{rhs}^{s, c} : T^{s, c} \quad \iota
\end{array}$$

$\text{STRUCTURES} \vdash I_{\text{ctor}}^{s, c}$ struct is a *structure constructor* if I has the single sort s , s has the single constructor c , no indices and no recursive fields, and $\vdash I^s$ large.

Define the projections of fields and the η -expansion as follows:

$$\begin{aligned}
\pi_f(x) &= I_{\text{rec}}^s. \{\bar{l}, v_f \{\bar{l}/\bar{u}\}\}(\bar{p}, \lambda z. A_f(\bar{p}, \pi_1(z), \dots, \pi_{f-1}(z)), \lambda \bar{a}. a_f, (), x) \\
\eta(x) &= I_{\text{ctor}}^{s, c}. \{\bar{l}\}(\bar{p}, \pi_1(x), \dots, \pi_m(x), ())
\end{aligned}$$

$$\begin{array}{c}
I \in \mathbf{E} \quad \vdash I_{\text{ctor}}^{s, c} \text{ struct} \quad \mathbf{E}[\Gamma] \vdash \bar{p} : \Gamma_{\text{param}} \quad \mathbf{E}[\Gamma] \vdash x : I^s. \{\bar{l}\}(\bar{p}, ()) \\
\mathbf{E}[\Gamma] \vdash \eta(x) : I^s. \{\bar{l}\}(\bar{p}, ()) \\
\hline
\mathbf{E}[\Gamma] \vdash \eta(x) \equiv x : I^s. \{\bar{l}\}(\bar{p}, ()) \quad \text{STRUCT-}\eta
\end{array}$$

$$\begin{array}{c}
\text{init_quot}(I_{\text{Eq}}) \in \mathbf{E} \quad \mathbf{E}[\Gamma] \vdash \alpha_1 \equiv \alpha_2 : \text{Sort}_l \\
\mathbf{E}[\Gamma] \vdash r_1 \equiv r_2 : \alpha_1 \rightarrow \alpha_1 \rightarrow \text{Prop} \\
\hline
\mathbf{E}[\Gamma] \vdash Q. \{l\}(\alpha_1, r_1) \equiv Q. \{l\}(\alpha_2, r_2) : \text{Sort}_l \quad \text{QUOT-FORM}
\end{array}$$

$$\begin{array}{c}
\text{init_quot}(I_{\text{Eq}}) \in \mathbf{E} \quad \mathbf{E}[\Gamma] \vdash \alpha_1 \equiv \alpha_2 : \text{Sort}_l \\
\mathbf{E}[\Gamma] \vdash r_1 \equiv r_2 : \alpha_1 \rightarrow \alpha_1 \rightarrow \text{Prop} \quad \mathbf{E}[\Gamma] \vdash a_1 \equiv a_2 : \alpha_1 \\
\hline
\mathbf{E}[\Gamma] \vdash Q_{\text{mk}}. \{l\}(\alpha_1, r_1, a_1) \equiv Q_{\text{mk}}. \{l\}(\alpha_2, r_2, a_2) : Q. \{l\}(\alpha_1, r_1) \quad \text{QUOT-INTRO}
\end{array}$$

$$\begin{array}{c}
\mathbf{init_quot}(I_{\text{Eq}}) \in \mathbf{E} \\
\mathbf{E}[\Gamma] \vdash \alpha_1 \equiv \alpha_2 : \text{Sort}_{l_1} \\
\mathbf{E}[\Gamma] \vdash r_1 \equiv r_2 : \alpha_1 \rightarrow \alpha_1 \rightarrow \text{Prop} \\
\mathbf{E}[\Gamma] \vdash \beta_1 \equiv \beta_2 : \text{Sort}_{l_2} \\
\mathbf{E}[\Gamma] \vdash f_1 \equiv f_2 : \alpha_1 \rightarrow \beta_1 \\
\mathbf{E}[\Gamma] \vdash h_1 \equiv h_2 : \forall a b : \alpha_1. r_1 a b \rightarrow I_{\text{Eq}}.\{l_2\}(\beta_1, f_1 a, f_1 b) \\
\mathbf{E}[\Gamma] \vdash a_1 \equiv a_2 : Q.\{l_1\}(\alpha_1, r_1) \\
\hline
\mathbf{E}[\Gamma] \vdash Q_{\text{lift}}.\{l_1, l_2\}(\alpha_1, r_1, \beta_1, f_1, h_1, a_1) \equiv Q_{\text{lift}}.\{l_1, l_2\}(\alpha_2, r_2, \beta_2, f_2, h_2, a_2) : \beta_1
\end{array} \text{QUOT-ELIM}$$

$$\begin{array}{c}
\mathbf{init_quot}(I_{\text{Eq}}) \in \mathbf{E} \\
\mathbf{E}[\Gamma] \vdash \alpha_1 \equiv \alpha_2 : \text{Sort}_l \\
\mathbf{E}[\Gamma] \vdash r_1 \equiv r_2 : \alpha_1 \rightarrow \alpha_1 \rightarrow \text{Prop} \\
\mathbf{E}[\Gamma] \vdash \beta_1 \equiv \beta_2 : Q.\{l\}(\alpha_1, r_1) \rightarrow \text{Prop} \\
\mathbf{E}[\Gamma] \vdash f_1 \equiv f_2 : \forall a : \alpha_1. \beta_1 Q_{\text{mk}}.\{l\}(\alpha_1, r_1, a) \\
\mathbf{E}[\Gamma] \vdash a_1 \equiv a_2 : Q.\{l\}(\alpha_1, r_1) \\
\mathbf{E}[\Gamma] \vdash \beta_1 a_1 \equiv \beta_2 a_2 : \text{Prop} \\
\hline
\mathbf{E}[\Gamma] \vdash Q_{\text{ind}}.\{l\}(\alpha_1, r_1, \beta_1, f_1, a_1) \equiv Q_{\text{ind}}.\{l\}(\alpha_2, r_2, \beta_2, f_2, a_2) : \beta_1 a_1
\end{array} \text{QUOT-IND}$$

$$\begin{array}{c}
\mathbf{init_quot}(I_{\text{Eq}}) \in \mathbf{E} \\
\mathbf{E}[\Gamma] \vdash \alpha : \text{Sort}_{l_1} \\
\mathbf{E}[\Gamma] \vdash r : \alpha \rightarrow \alpha \rightarrow \text{Prop} \\
\mathbf{E}[\Gamma] \vdash \beta : \text{Sort}_{l_2} \\
\mathbf{E}[\Gamma] \vdash f : \alpha \rightarrow \beta \\
\mathbf{E}[\Gamma] \vdash h : \forall a b : \alpha. r a b \rightarrow I_{\text{Eq}}.\{l_2\}(\beta, f a, f b) \\
\mathbf{E}[\Gamma] \vdash a : \alpha \\
\mathbf{E}[\Gamma] \vdash Q_{\text{lift}}.\{l_1, l_2\}(\alpha, r, \beta, f, h, Q_{\text{mk}}.\{l_1\}(\alpha, r, a)) : \beta \\
\mathbf{E}[\Gamma] \vdash f a : \beta \\
\hline
\mathbf{E}[\Gamma] \vdash Q_{\text{lift}}.\{l_1, l_2\}(\alpha, r, \beta, f, h, Q_{\text{mk}}.\{l_1\}(\alpha, r, a)) \equiv f a : \beta
\end{array} \text{QUOT-}\iota$$

$$\boxed{\mathbf{E}[\Delta] \vdash \sigma : \Gamma \quad (\text{well-typed substitutions})}$$

$$\frac{\forall (x : t) \in \Gamma. \mathbf{E}[\Delta] \vdash \sigma(x) : t[\sigma]}{\mathbf{E}[\Delta] \vdash \sigma : \Gamma} \qquad \frac{\forall (x : t) \in \Gamma. \mathbf{E}[\Delta] \vdash \sigma_1(x) \equiv \sigma_2(x) : t[\sigma_1]}{\mathbf{E}[\Delta] \vdash \sigma_1 \equiv \sigma_2 : \Gamma}$$

$$\boxed{\mathbf{E}[\Gamma] \vdash t \text{ type} \quad (\text{well-formedness of types})}$$

$$\frac{\mathbf{E}[\Gamma] \vdash t : \text{Sort}_u}{\mathbf{E}[\Gamma] \vdash t \text{ type}}$$

$E[\Gamma] \vdash t_1 \equiv t_2 \text{ type} \quad (\text{type equality}^6)$

$$\frac{E[\Gamma] \vdash t_1 \equiv t_2 : \text{Sort}_u}{E[\Gamma] \vdash t_1 \equiv t_2 \text{ type}} \qquad \frac{E[\Gamma] \vdash t_1 \equiv t_2 \text{ type} \quad E[\Gamma] \vdash t_2 \equiv t_3 \text{ type}}{E[\Gamma] \vdash t_1 \equiv t_3 \text{ type}}$$

$E \vdash \Gamma \text{ ctx} \quad (\text{well-formedness of contexts})$

$$\frac{}{E \vdash \cdot \text{ ctx}} \qquad \frac{E \vdash \Gamma \text{ ctx} \quad E[\Gamma] \vdash t \text{ type}}{E \vdash \Gamma, x : t \text{ ctx}}$$

$E[\Gamma] \vdash_P \Delta \text{ tele} \quad (\text{conditional well-formedness of telescopes})$

For a predicate P on levels,

$$\frac{}{E[\Gamma] \vdash_P \cdot \text{ tele}} \qquad \frac{E[\Gamma] \vdash_P \Delta \text{ tele} \quad E[\Gamma, \Delta] \vdash t : \text{Sort}_u \quad P(u)}{E[\Gamma] \vdash_P \Delta, x : t \text{ tele}}$$

Fix a mutual inductive block I with universe variables $\bar{u}$, parameters $(\bar{p} : \Gamma_{\text{param}})$, index telescopes Γ_{idx}^s and resulting universe Sort_l . Write $P_I(v) \iff \text{imax}(v, l) \leq l$.⁷

$E[\Gamma] \vdash_I (A, v) \text{ field} \quad (\text{well-formedness of (non-recursive) fields})$

$$\frac{E[\Gamma] \vdash A : \text{Sort}_v \quad P_I(v)}{E[\Gamma] \vdash_I (A, v) \text{ field}}$$

where (A, v) is an ordinary field with its level.

$E[\Gamma] \vdash_I R \text{ recfield} \quad (\text{well-formedness of recursive fields})$

$$\frac{E[\Gamma] \vdash_{P_I} \Theta \text{ tele} \quad E[\Gamma, \Theta] \vdash \bar{j} : \Gamma_{\text{idx}}^s}{E[\Gamma] \vdash_I \Theta \rightarrow I^s.\{\bar{u}\}(\bar{p}, \bar{j}) \text{ recfield}}$$

$E \vdash_I I_{\text{ctor}}^{s,c} \text{ ctor} \quad (\text{well-formedness of constructors})$

For $I_{\text{ctor}}^{s,c}$ with ordinary fields $(a_1 : A_1), \dots, (a_m : A_m)$ at levels $v_1, \dots, v_m$, recursive fields $R_1, \dots, R_k$ and target indices $\bar{i}$,

⁶We can't use the transitivity of definitional equality here, because three types may be pairwise equal but at different universe sorts. Only with uniqueness of sorts, proved much later, can we conclude that they are really the same.

⁷This is *more* permissive than Lean, but it is still sound.

$$\begin{array}{c}
\frac{\forall f. E[\Gamma_{\text{param}}, a_1 : A_1, \dots, a_{f-1} : A_{f-1}] \vdash_I (A_f, v_f) \text{ field} \quad \forall g. E[\Gamma_{\text{param}}, \Gamma_{\text{field}}^{s,c}] \vdash_I R_g \text{ recfield} \quad E[\Gamma_{\text{param}}, \Gamma_{\text{field}}^{s,c}] \vdash \bar{i} : \Gamma_{\text{idx}}^s}{E \vdash_I I_{\text{ctor}}^{s,c} \text{ ctor}} \\
\\
\boxed{E \vdash I \text{ inductive} \quad (\text{well-formedness of inductive blocks})} \\
\\
\frac{E[\cdot] \vdash \Gamma_{\text{param}} \text{ tele} \quad \forall s. E[\Gamma_{\text{param}}] \vdash \Gamma_{\text{idx}}^s \text{ tele} \quad \forall s, c. E \vdash_I I_{\text{ctor}}^{s,c} \text{ ctor}}{E \vdash I \text{ inductive}}
\end{array}$$

Definition 13 (Equality).

inductive $\text{Eq}.\{u\} (\alpha : \text{Sort}_u) (a : \alpha) : (b : \alpha) \rightarrow \text{Prop}$ **where**
 $\mid \text{refl} : \text{Eq}.\{u\}(\alpha, a, a)$

$$\begin{array}{c}
\boxed{E \vdash D \text{ entry} \quad (\text{well-formedness of environment entries})} \\
\\
\frac{E[\cdot] \vdash t \text{ type}}{E \vdash \mathbf{axiom} \ c.\{\bar{u}\} : t \text{ entry}} \quad \frac{E[\cdot] \vdash t \text{ type} \quad E[\cdot] \vdash e : t}{E \vdash \mathbf{opaque} \ c.\{\bar{u}\} : t \text{ entry}} \quad \frac{E[\cdot] \vdash t \text{ type} \quad E[\cdot] \vdash e : t}{E \vdash \mathbf{def} \ c.\{\bar{u}\} : t := e \text{ entry}} \\
\\
\frac{E \vdash I \text{ inductive}}{E \vdash \mathbf{mutual inductive} \ I \dots \mathbf{end} \text{ entry}} \quad \frac{E(I_{\text{Eq}}) = \text{Eq}}{E \vdash \mathbf{init_quot}(I_{\text{Eq}}) \text{ entry}} \\
\\
\boxed{\vdash E \text{ env} \quad (\text{well-formedness of environments})} \\
\\
\frac{}{\vdash \cdot \text{ env}} \quad \frac{\vdash E \text{ env} \quad E \vdash D \text{ entry}}{\vdash E, D \text{ env}}
\end{array}$$

3 Metatheory

Lemma 14 (Regularity). *If $E[\Gamma] \vdash e_1 \equiv e_2 : t$, then $E[\Gamma] \vdash e_1 : t$, $E[\Gamma] \vdash e_2 : t$ and $E[\Gamma] \vdash t \text{ type}$.*

Proof. SYMM, TRANS, and induction on the typing derivation. $\square$

Lemma 15 (Environment extension). *Let E be a prefix of E' . If $E[\Gamma] \vdash e_1 \equiv e_2 : t$, then $E'[\Gamma] \vdash e_1 \equiv e_2 : t$, and if $E \vdash \Gamma \text{ ctx}$, then $E' \vdash \Gamma \text{ ctx}$.*

Proof. The first is by induction on the typing derivation of $E[\Gamma] \vdash e_1 \equiv e_2 : t$. The second is by induction on the derivation of $E \vdash \Gamma \text{ ctx}$, using the first. $\square$

Lemma 16 (Level instantiation). *Let $\bar{l}$ be levels. If $E[\Gamma] \vdash e_1 \equiv e_2 : t$, then $E[\Gamma\{\bar{l}/\bar{u}\}] \vdash e_1\{\bar{l}/\bar{u}\} \equiv e_2\{\bar{l}/\bar{u}\} : t\{\bar{l}/\bar{u}\}$, and if $E \vdash \Gamma \text{ ctx}$, then $E \vdash \Gamma\{\bar{l}/\bar{u}\} \text{ ctx}$.*

Proof. The first is by induction on the typing derivation of $E[\Gamma] \vdash e_1 \equiv e_2 : t$. The second is by induction on the derivation of $E \vdash \Gamma \text{ ctx}$, using the first. $\square$

Lemma 17 (Weakening). *If $E[\Gamma_1, \Gamma_2] \vdash e_1 \equiv e_2 : t$ and y is fresh, then $E[\Gamma_1, y : t', \Gamma_2] \vdash e_1 \equiv e_2 : t$.*

Lemma 18 (Substitution). *If $E[\Delta] \vdash \sigma : \Gamma$ and $E[\Gamma] \vdash e_1 \equiv e_2 : t$, then $E[\Delta] \vdash e_1[\sigma] \equiv e_2[\sigma] : t[\sigma]$.*

Proof. Weakening and substitution are proved together, by induction on the typing derivation of $E[\Gamma] \vdash e_1 \equiv e_2 : t$, for substitutions sending each variable either to a variable of the same type or to a term of the substituted type. $\square$

Lemma 19 (Context conversion). *If $E[\Gamma] \vdash t_1 \equiv t_2 : \text{Sort}_l$ and $E[\Gamma, x : t_1] \vdash e_1 \equiv e_2 : t$, then $E[\Gamma, x : t_2] \vdash e_1 \equiv e_2 : t$.*

Proof. The identity substitution from $\Gamma, x : t_1$ to $\Gamma, x : t_2$ is of the form above, typing x by CONV. $\square$

Lemma 20 (Substitution congruence). *If $E \vdash \Gamma \text{ ctx}$, $E[\Delta] \vdash \sigma_1 \equiv \sigma_2 : \Gamma$ and $E[\Gamma] \vdash e_1 \equiv e_2 : t$, then $E[\Delta] \vdash e_1[\sigma_1] \equiv e_2[\sigma_2] : t[\sigma_1]$.*

Proof. By induction on the derivation of $E \vdash \Gamma \text{ ctx}$, using substitution and $\forall$ - β . $\square$

Theorem 21 (Definitional inversion). *Suppose $\vdash E \text{ env}$ and $E \vdash \Gamma \text{ ctx}$.*

1. (Injectivity of sorts) *If $E[\Gamma] \vdash \text{Sort}_{l_1} \equiv \text{Sort}_{l_2} \text{ type}$, then $l_1 = l_2$.*
2. (Injectivity of $\forall$) *If $E[\Gamma] \vdash \forall x : t_1. t'_1 \equiv \forall x : t_2. t'_2 \text{ type}$, then $E[\Gamma] \vdash t_1 \equiv t_2 \text{ type}$ and $E[\Gamma, x : t_1] \vdash t'_1 \equiv t'_2 \text{ type}$.*
3. (Injectivity of inductive types) *If $E[\Gamma] \vdash I^s.\{\bar{l}_1\}(\bar{p}_1, \bar{i}_1) \equiv I^s.\{\bar{l}_2\}(\bar{p}_2, \bar{i}_2) : \text{Sort}_{l_I\{\bar{l}_1/\bar{u}\}}$, then $\bar{l}_1 = \bar{l}_2$, and each $p_{1,k} \equiv p_{2,k}$ and each $i_{1,k} \equiv i_{2,k}$ holds in $E[\Gamma]$ at some type.*
4. (Injectivity of quotient types) *If $E[\Gamma] \vdash Q.\{l_1\}(\alpha_1, r_1) \equiv Q.\{l_2\}(\alpha_2, r_2) \text{ type}$, then $l_1 = l_2$, $E[\Gamma] \vdash \alpha_1 \equiv \alpha_2 \text{ type}$ and $E[\Gamma] \vdash r_1 \equiv r_2 : \alpha_1 \rightarrow \alpha_1 \rightarrow \text{Prop}$.*
5. (No-confusion) *Types headed by different type formers, and inductive types of different blocks or sorts, are not type-equal.*

Proof. Adequacy of the domain model! $\square$

Corollary 22 (Equational consistency). *Not all definitional equalities hold.*

Proof. By no-confusion, e.g. $\text{Prop} \not\equiv \text{Prop} \rightarrow \text{Prop}$. $\square$

Theorem 23 (Uniqueness of typing). *Suppose $\vdash E \text{ env}$ and $E \vdash \Gamma \text{ ctx}$. If $E[\Gamma] \vdash e : t_1$ and $E[\Gamma] \vdash e : t_2$, then $E[\Gamma] \vdash t_1 \equiv t_2 \text{ type}$.*

Proof. Assume $E[\Gamma] \vdash e : t_1$ and generalise to $E[\Gamma] \vdash e_1 \equiv e_2 : t$, then every type of e_1 or of e_2 is type-equal to t , by induction on the typing derivation of $E[\Gamma] \vdash e_1 \equiv e_2 : t$. Use syntactic inversion and the injectivity clauses of definitional inversion. $\square$

Theorem 24 (Uniqueness of sorts). *Suppose $\vdash E$ env and $E \vdash \Gamma$ ctx. If $E[\Gamma] \vdash t_1 \equiv t_2$ type, then $E[\Gamma] \vdash t_1 \equiv t_2 : \text{Sort}_l$ for some l .*

Proof. By induction on the derivation of $E[\Gamma] \vdash t_1 \equiv t_2$ type, recall it is defined as the transitive closure of $E[\Gamma] \vdash t_1 \equiv t_2 : \text{Sort}_u$. In the transitive case, the middle type has two sorts Sort_{l_1} and Sort_{l_2} , so apply uniqueness of typing and injectivity of sorts to get $l_1 = l_2$. $\square$

$$\boxed{E \vdash e_1 \rightsquigarrow e_2 \quad (\text{weak-head reduction})}$$

Elimination frames:

$$K \in \text{Frame} ::= \square a \mid I_{\text{rec}}^s \cdot \{\bar{l}, v\}(\bar{p}, \bar{\mu}, \bar{m}, \bar{i}, \square) \mid Q_{\text{lift}} \cdot \{l_1, l_2\}(\alpha, r, \beta, f, h, \square) \mid Q_{\text{ind}} \cdot \{l\}(\alpha, r, \beta, f, \square)$$

$$\begin{array}{c} \frac{E \vdash e_1 \rightsquigarrow e_2}{E \vdash K[e_1] \rightsquigarrow K[e_2]} \text{FRAME} \qquad \frac{}{E \vdash (\lambda x : t.e) a \rightsquigarrow e[a/x]} \beta \\[2ex] \frac{}{E \vdash \text{let } E \ x : t := v; e \rightsquigarrow e[v/x]} \zeta \qquad \frac{(\text{def } c.\{\bar{u}\} : t := e) \in E}{E \vdash c.\{\bar{l}\} \rightsquigarrow e\{\bar{l}/\bar{u}\}} \delta \\[2ex] \frac{}{E \vdash I_{\text{rec}}^s \cdot \{\bar{l}_1, v\}(\bar{p}_1, \bar{\mu}, \bar{m}, \bar{i}, I_{\text{ctor}}^{s,c} \cdot \{\bar{l}_2\}(\bar{p}_2, \bar{a}, \bar{r})) \rightsquigarrow \text{rhs}^{s,c}} \iota \\[2ex] \frac{}{E \vdash Q_{\text{lift}} \cdot \{l_1, l_2\}(\alpha, r, \beta, f, h, Q_{\text{mk}} \cdot \{l'_1\}(\alpha', r', a)) \rightsquigarrow f a} \text{QUOT-}\iota \\[2ex] \frac{}{E \vdash Q_{\text{ind}} \cdot \{l\}(\alpha, r, \beta, f, Q_{\text{mk}} \cdot \{l'\}(\alpha', r', a)) \rightsquigarrow f a} \text{QUOT-IND-}\iota \end{array}$$

where $\text{rhs}^{s,c}$ is taken at the levels $\bar{l}_1$ and parameters $\bar{p}_1$ of the recursor. Write $E \vdash e_1 \rightsquigarrow^* e_2$ for the reflexive transitive closure.

Theorem 25 (Subject reduction). *Suppose $\vdash E$ env and $E \vdash \Gamma$ ctx. If $E[\Gamma] \vdash e_1 : t$ and $E \vdash e_1 \rightsquigarrow^* e_2$, then $E[\Gamma] \vdash e_1 \equiv e_2 : t$.*

Proof. By induction on the reduction, it suffices to treat one step, by induction on its derivation.

- FRAME: congruence
- β : injectivity of $\forall$
- ι : injectivity of inductive types

$\square$

Proposition 26 (K-like reduction). *Suppose $\vdash E \text{ env}$ and $E \vdash \Gamma \text{ ctx}$. If $E[\Gamma] \vdash I^s.\{\bar{l}\}(\bar{p}, \bar{i}) : \text{Prop}$, $E[\Gamma] \vdash x : I^s.\{\bar{l}\}(\bar{p}, \bar{i})$, $E[\Gamma] \vdash I_{\text{ctor}}^{s,c}.\{\bar{l}\}(\bar{p}, \bar{a}, \bar{r}) : I^s.\{\bar{l}\}(\bar{p}, \bar{i})$ and $E[\Gamma] \vdash I_{\text{rec}}^s.\{\bar{l}, v\}(\bar{p}, \bar{\mu}, \bar{m}, \bar{i}, x) : t$, then*

$$E[\Gamma] \vdash I_{\text{rec}}^s.\{\bar{l}, v\}(\bar{p}, \bar{\mu}, \bar{m}, \bar{i}, x) \equiv \text{rhs}^{s,c} : t.$$

Proof. By PF-IRREL, $x \equiv I_{\text{ctor}}^{s,c}.\{\bar{l}\}(\bar{p}, \bar{a}, \bar{r})$, so by IND-ELIM the recursor on x is equal to the recursor on the constructor, which is equal to $\text{rhs}^{s,c}$ by subject reduction for ι . $\square$

Proposition 27 (Unit-like eta). *Suppose $\vdash E \text{ env}$ and $E \vdash \Gamma \text{ ctx}$, $I \in E$, $\vdash I_{\text{ctor}}^{s,c} \text{ struct}$ and $I_{\text{ctor}}^{s,c}$ has no fields. If $E[\Gamma] \vdash \bar{p} : \Gamma_{\text{param}}$, $E[\Gamma] \vdash x_1 : I^s.\{\bar{l}\}(\bar{p}, ())$ and $E[\Gamma] \vdash x_2 : I^s.\{\bar{l}\}(\bar{p}, ())$, then $E[\Gamma] \vdash x_1 \equiv x_2 : I^s.\{\bar{l}\}(\bar{p}, ())$.*

Proof. Since the constructor has no fields, $\eta(x_1)$ and $\eta(x_2)$ are both $I_{\text{ctor}}^{s,c}.\{\bar{l}\}(\bar{p}, (), ())$, which is well-typed by well-formedness of I . By STRUCT- η , $x_1 \equiv \eta(x_1) = \eta(x_2) \equiv x_2$. $\square$

4 Soundness of the checker

5 Domain model

6 Gluing model

7 Consistency

(TODO)

Bibliography

- [1] M. Carneiro, T. Coquand, A. Frabetti Mathieu, M. Lennon-Bertrand, P.-A. Melliès, and S. Weirich, “Definitional Inversion, Without Normalisation.” [Online]. Available: <https://arxiv.org/abs/2607.13662>
- [2] M. Carneiro, “The type theory of Lean,” *preparation* (<https://github.com/digama0/lean-type-theory/releases>), 2019.
- [3] P. Dybjer, “Inductive Families,” *Formal Aspects of Computing*, vol. 6, pp. 440–465, 1994, doi: 10.1007/BF01211308.
- [4] L. de Moura and S. Ullrich, “The Lean 4 Theorem Prover and Programming Language,” in *Automated Deduction – CADE 28*, A. Platzer and G. Sutcliffe, Eds., Springer, 2021, pp. 625–635. doi: 10.1007/978-3-030-79876-5_37.
- [5] M. Carneiro, “Lean4Lean: Towards a Formalized Metatheory for the Lean Theorem Prover.” 2024. doi: 10.48550/arXiv.2403.14064.

- [6] M. Lennon-Bertrand, “What Does It Take to Certify a Conversion Checker?,” in *10th International Conference on Formal Structures for Computation and Deduction (FSCD 2025)*, in Leibniz International Proceedings in Informatics (LIPIcs). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025, p. 27:1–27:23. doi: 10.4230/LIPIcs.FSCD.2025.27.
- [7] T. Lamiaux, Y. Forster, M. Sozeau, and N. Tabareau, “Nested Inductive Types: Justified and Usable Nested Inductive Types in Lean and Rocq.” [Online]. Available: <https://hal.science/hal-05366368>
- [8] S. Awodey, “Natural Models of Homotopy Type Theory,” *Mathematical Structures in Computer Science*, vol. 28, no. 2, pp. 241–286, 2018, doi: 10.1017/S0960129516000268.
- [9] T. Coquand and C. Paulin-Mohring, “Inductively Defined Types,” in *COLOG-88*, P. Martin-Löf and G. Mints, Eds., in Lecture Notes in Computer Science, vol. 417. Springer, 1990, pp. 50–66. doi: 10.1007/3-540-52335-9_47.
- [10] C. Paulin-Mohring, “Inductive Definitions in the System Coq: Rules and Properties,” in *Typed Lambda Calculi and Applications*, M. Bezem and J. F. Groote, Eds., in Lecture Notes in Computer Science, vol. 664. Springer, 1993.